

DOCUMENTATION

Mohd Shahid

1. Functions (def keyword)

Functions let you create reusable blocks of code.

Basic Syntax

```
def greet():  
    print("Hello!")
```

Calling the function:

```
greet()
```

Output:

Hello!

Function with Inputs

```
def greet(name):  
    print("Hello", name)
```

```
greet("Aman")
```

Output:

Hello Aman

2. Function Parameters

Parameters allow data to be passed into functions.

Positional Parameters

Arguments are matched by position.

```
def add(a, b):  
    print(a + b)
```

```
add(5, 3)
```

Output:

8

Keyword Parameters

Arguments are matched by name.

```
def student(name, age):  
    print(name, age)
```

```
student(age=20, name="Riya")
```

Output:

Riya 20

Default Parameters

Provide default values.

```
def greet(name="Guest"):  
    print("Hello", name)
```

```
greet()  
greet("Arjun")
```

Output:

Hello Guest
Hello Arjun

3. Return Values

Functions can send results back using return.

```
def square(num):  
    return num * num
```

```
result = square(4)  
print(result)
```

Output:

16

A function can return multiple values:

```
def calc(a, b):  
    return a + b, a - b
```

```
sum_value, diff_value = calc(10, 5)
```

```
print(sum_value)
```

```
print(diff_value)
```

4. Dictionaries

Dictionaries store data as key-value pairs.

Creating a Dictionary

```
student = {  
    "name": "Rahul",  
    "age": 21,  
    "course": "Python"  
}
```

Accessing Values

```
print(student["name"])
```

Output:

Rahul

Adding/Updating Data

```
student["age"] = 22  
student["city"] = "Delhi"
```

Looping Through Dictionary

```
for key, value in student.items():  
    print(key, value)
```

5. String Methods

Python provides many built-in string methods.

Common String Methods

```
text = "python programming"
```

Uppercase

```
print(text.upper())
```

Output:

PYTHON PROGRAMMING

Capitalize

```
print(text.capitalize())
```

Replace

```
print(text.replace("python", "Java"))
```

Split

```
print(text.split())
```

Find

```
print(text.find("program"))
```

Check Methods

```
print(text.islower())
```

```
print("123".isdigit())
```

6. Scope (Local vs Global Variables)

Scope determines where variables can be accessed.

Local Variable

Defined inside a function.

```
def test():
```

```
    x = 10
```

```
    print(x)
```

```
test()
```

x exists only inside the function.

Global Variable

Defined outside functions.

```
x = 100
```

```
def show():
```

```
print(x)
```

```
show()
```

Output:

```
100
```

Using global

```
count = 0
```

```
def increase():  
    global count  
    count += 1
```

```
increase()  
print(count)
```

Output:

```
1
```

7. Built-in Functions

Python has many powerful built-in functions.

Common Examples

len()

```
print(len("Python"))
```

Output:

```
6
```

type()

```
print(type(10))
```

max() and min()

```
numbers = [4, 8, 1, 9]
```

```
print(max(numbers))
```

```
print(min(numbers))
```

sum()

```
print(sum(numbers))
```

sorted()

```
print(sorted(numbers))
```

range()

```
for i in range(5):
```

```
    print(i)
```

enumerate()

```
fruits = ["apple", "banana"]
```

```
for index, fruit in enumerate(fruits):
```

```
    print(index, fruit)
```

Mini Practice Program

```
def student_info(name, marks=0):
```

```
    result = {
```

```
        "name": name,
```

```
        "marks": marks,
```

```
        "status": "Pass" if marks >= 40 else "Fail"
```

```
    }
```

```
    return result
```

```
student = student_info("Aisha", 75)
```

```
print(student)
```

```
print(student["name"].upper())
```

Output:

```
{'name': 'Aisha', 'marks': 75, 'status': 'Pass'}
```

```
AISHA
```

We use cookies

Cookies help this site function, measure usage, and support marketing.

1. Functions for Common Calculations

Area Calculations

```
def area_rectangle(length, width):  
    return length * width
```

```
def area_circle(radius):  
    return 3.14 * radius * radius
```

```
print(area_rectangle(5, 4))  
print(area_circle(7))
```

Volume Calculations

```
def volume_cube(side):  
    return side ** 3
```

```
def volume_cylinder(radius, height):  
    return 3.14 * radius * radius * height
```

```
print(volume_cube(3))  
print(volume_cylinder(2, 5))
```

Conversion Functions

```
def celsius_to_fahrenheit(c):  
    return (c * 9/5) + 32  
  
def km_to_miles(km):  
    return km * 0.621  
  
print(celsius_to_fahrenheit(25))  
print(km_to_miles(10))
```

2. Currency Converter

```
def usd_to_inr(usd):  
    return usd * 83  
  
def inr_to_usd(inr):  
    return inr / 83  
  
def eur_to_inr(eur):  
    return eur * 90  
  
print("100 USD =", usd_to_inr(100), "INR")  
print("8300 INR =", inr_to_usd(8300), "USD")  
print("50 EUR =", eur_to_inr(50), "INR")
```

You can also build a menu-based converter:

```
def currency_converter():  
    amount = float(input("Enter amount: "))  
    choice = input("Convert USD to INR? (yes/no): ")  
  
    if choice.lower() == "yes":  
        print("Converted:", amount * 83)  
  
currency_converter()
```

3. Student Database Using Dictionaries

```
students = {  
    101: {"name": "Aman", "marks": 85},  
    102: {"name": "Riya", "marks": 90},  
    103: {"name": "Karan", "marks": 78}
```



```
}
```

```
print(students[101])
```

```
for roll, data in students.items():  
    print("Roll:", roll)  
    print("Name:", data["name"])  
    print("Marks:", data["marks"])
```

Adding a Student

```
students[104] = {"name": "Neha", "marks": 88}
```

```
print(students)
```

4. Text Analyzer

```
def text_analyzer(text):  
    words = len(text.split())  
    characters = len(text)  
  
    vowels = 0  
    for char in text.lower():  
        if char in "aeiou":  
            vowels += 1  
  
    print("Words:", words)  
    print("Characters:", characters)  
    print("Vowels:", vowels)
```

```
text_analyzer("Python is amazing")
```

Output:

```
Words: 3  
Characters: 17  
Vowels: 6
```

5. Simple Bank Account System

```
balance = 1000
```

```
def deposit(amount):
    global balance
    balance += amount
    print("Deposited:", amount)
    print("New Balance:", balance)

def withdraw(amount):
    global balance

    if amount <= balance:
        balance -= amount
        print("Withdrawn:", amount)
    else:
        print("Insufficient balance")

    print("Current Balance:", balance)

deposit(500)
withdraw(300)
withdraw(2000)
```

6. Practice String Methods

```
text = " Python Programming "
```

upper()

```
print(text.upper())
```

lower()

```
print(text.lower())
```

strip()

Removes extra spaces.

```
print(text.strip())
```

split()

```
print(text.split())
```

More Useful String Methods

```
print(text.replace("Python", "Java"))  
print(text.startswith(" "))  
print(text.endswith(" "))
```

7. Functions with Different Parameter Types

Positional Parameters

```
def add(a, b):  
    print(a + b)
```

```
add(5, 3)
```

Keyword Parameters

```
def student(name, age):  
    print(name, age)
```

```
student(age=20, name="Riya")
```

Default Parameters

```
def greet(name="Guest"):  
    print("Hello", name)
```

```
greet()  
greet("Aman")
```

Variable-Length Arguments (*args)

```
def total(*numbers):  
    print(sum(numbers))
```

```
total(1, 2, 3)  
total(10, 20, 30, 40)
```

Keyword Variable Arguments (**kwargs)

```
def profile(**data):
    for key, value in data.items():
        print(key, ":", value)

profile(name="Aisha", age=22, city="Delhi")
```

Combined Mini Project

```
students = {}

def add_student(roll, name, marks):
    students[roll] = {
        "name": name,
        "marks": marks
    }

def show_students():
    for roll, info in students.items():
        print(roll, info)

def find_average():
    total = 0

    for info in students.values():
        total += info["marks"]

    average = total / len(students)
    print("Average Marks:", average)

add_student(1, "Aman", 85)
add_student(2, "Riya", 92)

show_students()
find_average()
```

We use cookies

Cookies help this site function, measure usage, and support marketing.